

05 - 10.
일반화
프로그래밍

일반화 프로그래밍

일반화 프로그래밍

- 데이터 타입에 의존하지 않고, 알고리즘이나 로직을 일반화하여 작성하는 프로그래밍 패러다임을 일반화 프로그래밍(Generic Programming)이라 한다.
 - 기능은 같으나 타입만 다른 코드의 중복을 제거한다.
- 매개변수를 통해 타입을 전달한다.
 - 사용할 때, 타입을 결정하는 것이다.
- C#에서는 제네릭(Generic)으로 제공된다.

제네릭

타입 파라미터

- 타입 파라미터(Type Parameter)는 위치지정자 역할을 수행한다.
 - 즉, 프로그래머가 명시한 타입 인수로 대체된다.
- 타입 파라미터의 이름은 T라는 접두사를 붙인다.[1]
- 클래스, 인터페이스, 대리자, 메소드에 붙일 수 있다.

```
public void Swap<T>(ref T lhs, ref T rhs)
{
    T temp = lhs;
    lhs = rhs;
    rhs = temp;
}
```

```
// 사용할 때는 실제 대체할 타입을 명시한다.
// 컴파일러가 추론할 수 있는 경우 굳이 적지 않아도 된다.
int a = 10, b = 20;
Swap<int>(ref a, ref b);
```

제약

- 타입 인수가 반드시 가져야 하는 조건을 명시할 수 있는데 이를 제약(Constraint)이라 한다.
- 제약 조건을 만족하지 않으면 컴파일 오류가 발생한다.

```
// 아무 타입이나 넣을 수 없고, 구조체 타입만 가능하다.  
public void Foo<T>(T item) where T : struct  
{  
    // ~  
}
```

동작 원리

- 제네릭은 주어진 타입에 따라 실행 시 동작이 다르다.
 - 값 타입인 경우 각 타입별로 독립적인 코드가 생성된다.[1]
 - 참조 타입인 경우 하나의 코드로 공유한다.
- 코드 생성은 실제로 사용[2]할 때 생성되며, 단 한 번만 생성된다.

[1]: 특정 타입 버전의 코드를 생성하는 것을 **특수화**(Specialization)라고 한다.

[2]: 이런 특성을 지연(Lazy)됐다고 표현한다.

부록

더 나아가기

- 일반화 프로그래밍의 정의와 이를 통해 얻을 수 있는 가장 큰 이점은 무엇인가요?

참고자료

- [Generics in .NET - .NET | Microsoft Learn](#)
- [Generics in the runtime - C# | Microsoft Learn](#)